

Sensoren und Aktoren des elektrischen Tischförderbands

Eine Auswahl

Sven Hein, Aiko Cassens, Jenik Kröger, Jan Ahrens, Bjarne Rieken
Janssen

13. April 2026

- 1 Einleitung
- 2 Specific Sensor/Actor
- 3 externer Code
- 4 Doxygen
- 5 Test
- 6 Reflex-Lichtschanke E18-D80N
- 7 Bibliothek
- 8 Beispiel

9 Kalibrierung

10 Einfache Anwendung

11 Quellen

Einleitung

Allgemeines I

Grundsätzlich: Sensor (lat: „Fühler“) dienen zur **qualitativen** und **quantitativen Auffassung** von physikalischen, chemischen, biologischen, klimatischen und medizinischen Größen. [HEr23]

Jedes Sensorsystem besteht aus folgenden Komponenten:

- **Sensor-Element** z.B. veränderlicher Widerstand
- **Auswerte-Elektronik** z.B. Mikrocontroller

Quellen: [LM12], [Rod23]

Literatur I

Folgende Literatur wurde verwendet:

- ① Einführung in die Mechatronik, Werner Roddeck, Springer Fachmedien Wiesbaden.
Beschreibung: Dieses Fachbuch beschreibt die Funktionsweise unterschiedlicher Komponenten der Mechatronik, auch Optische Sensoren.
- ② Sensoren und Aktoren, Jörg Wallaschek, Springer Berlin Heidelberg.
Beschreibung: Dieses Buch geht insbesondere auf die Schnittstellen zwischen Sensoren und Aktoren ein und beleuchtet hauptsächlich die Ansteuerung und Wirkprinzipien dieser Systeme.
- ③ Datenblatt E18-D80NK, Roboter-Bausatz.de
- ④ Bounce2 Repository von Thomas Fredericks

Weitere verwendete Quellen finden sich im Quellenverzeichnis.

```
#include <Bounce2.h>
```

```
// Pins definieren
```

```
#define LICH_SCH_PIN 2
```

```
#define LED_PIN 13
```

```
// Bounce-objekt zur Entprellung des Eingangssignals
```

```
// Die Klasse Bounce aus der Bibliothek Bounce2 sorgt da
```

```
// dass Stoerungen (Prellen) am Eingangssignal gefiltert
```

```
Bounce debouncer = Bounce();
```

```
// Initialisierung der Hardware und seriellen Kommunikation
```

```
void setup() {
```

```
    pinMode(LICH_SCH_PIN, INPUT_PULLUP); // Eingang mit
```

```
    pinMode(LED_PIN, OUTPUT); // LED als Aus
```

```
    debouncer.attach(LICH_SCH_PIN); // Pin dem Deb
```

```
    debouncer.interval(50); // 50 ms Entp
```

```
Serial.begin(9600);           // Start Serial
}

// Hauptprogrammschleife
// Prueft kontinuierlich den Zustand der Lichtschranke
// und steuert die LED sowie die serielle Ausgabe.
void loop() {
    debouncer.update(); // Entprellung aktualisieren

    if (debouncer.read() == LOW) {
        digitalWrite(LED_PIN, HIGH); // LED an
        Serial.println(" Objekt-erkannt!"); // Ausgabe in
    } else {
        digitalWrite(LED_PIN, LOW); // LED aus
    }
}

#include <Bounce2.h>
```



```
/**  
 * @file codeTest.ino  
 * @brief Steuerung eines Lichtschranken-Sensors mit Ent  
 *  
 * Dieses Programm liest das Signal einer Lichtschranke  
 * entprellt es mithilfe der Bounce2-Bibliothek und steuert  
 * eine LED entsprechend. Zusätzlich erfolgt eine Ausgabe  
 * ueber den Serial-Monitor.  
 */
```

```
// Pins definieren
```

```
/**  
 * @def LICH_SCH_PIN  
 * @brief GPIO-Pin fuer das Signal der Lichtschranke.  
 */  
#define LICH_SCH_PIN 2
```

```
/**
```

```
* @def LED_PIN
* @brief GPIO-Pin fuer die LED-Ausgabe.
*/
#define LED_PIN 13

/**
* @brief Bounce-Objekt zur Entprellung des Eingangssign
*
* Die Klasse Bounce aus der Bibliothek Bounce2 sorgt da
* dass Stoerungen (Prellen) am Eingangssignal gefiltert
*/
Bounce debouncer = Bounce();

/**
* @brief Initialisierung der Hardware und seriellen Kon
*/
void setup() {

    pinMode(LICH_SCH_PIN, INPUT_PULLUP); ///< Eingang n
```

```
pinMode(LED_PIN, OUTPUT);

debouncer.attach(LICH_SCH_PIN);
debouncer.interval(50);

Serial.begin(9600);

}

/**
 * @brief Hauptprogrammschleife.
 *
 * Prueft kontinuierlich den Zustand der Lichtschranke
 * und steuert die LED sowie die serielle Ausgabe.
 */
void loop() {

    debouncer.update();

    if (debouncer.read() == LOW) {
```

///< LED als A

///< Pin dem D

///< 50 ms En

///< Start Ser

```
digitalWrite(LED_PIN, HIGH);    ///LED an  
  
Serial.println(" Objekt-erkannt!");    ///Ausgabe  
} else {  
    digitalWrite(LED_PIN, LOW);    ///LED aus  
}  
}
```

Test der Lichtschranke - Beschreibung I

Auf dem Arduino soll die eingebaute LED dauerhaft leuchten. Die Lichtschranke wird ebenfalls ordnungsgemäß angeschlossen. Nun soll die ordnungsgemäße Funktion der Lichtschranke geprüft werden. Folgendes Vorgehen wird nachgehend beschrieben.

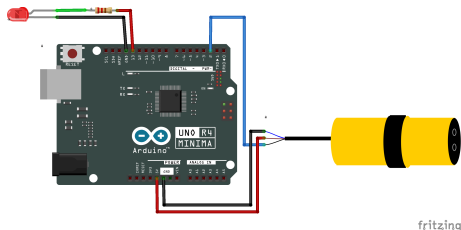


Abbildung: Anschlussplan des Sensors mit LED

Test der Lichtschranke - Ablauf I

- ➊ Starten der ArduinoIDE
- ➋ Verbinden der Lichtschranke an die Versorgungspins (5V, GND) des Mikrocontrollers
- ➌ Verbinden der Signalleitung der Lichtschranke an einen GPIO Pin des Mikrocontrollers
- ➍ Verbinden des Mikrocontrollers mit dem Rechner per USB
- ➎ Öffnen der ArduinoIDE
- ➏ Öffnen des Sketches zur Erprobung
- ➐ Eintragen des mit der Signalleitung verbundenen GPIO Pins an der vorgesehenen Stelle
- ➑ Sketch auf den Mikrocontroller spielen

Spezifikation Reflex-Lichtschranke E18-D80N I

Tabelle: Lichtschranke Spezifikation

Parameter	Wert/Detail
Modell	Funduino E18-D80NK
Betriebsspannung	5V DC
Stromaufnahme	ca. 100mA
Einstellbereich	3cm - 80cm
Reaktionszeit	kleiner 2ms
Erfassungswinkel	bis zu 15°
Temperaturbereich	-25°C bis +55°C
Maße	Ø 17,6mm x 50mm

Technische Darstellung der Lichtschranke I

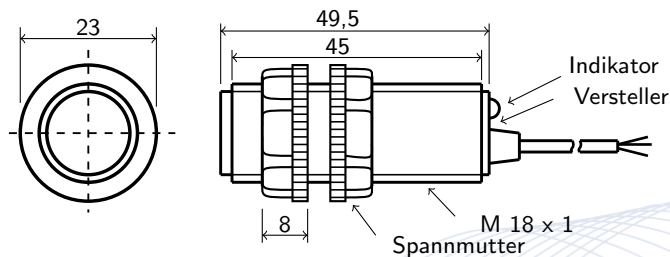


Abbildung: Mechanische Darstellung (vollständig)

Hinweise über den Sensor I

Hauptfunktionen

Der E18-D80NK fungiert als optische **Lichtschranke** zur berührungslosen Objekterkennung. Die Hauptmerkmale sind die digitale Signalausgabe, die Unempfindlichkeit gegenüber sichtbarem Licht und die kompakte Bauweise für die Integration in Steuerungssysteme mit einem **Arduino**.

Hinweise über den Sensor II

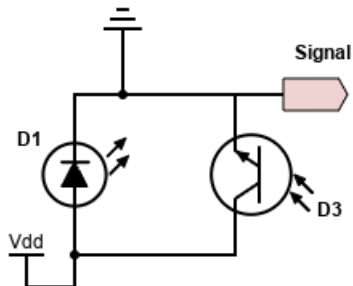


Abbildung: Schaltplan des Sensors E18-D80NK

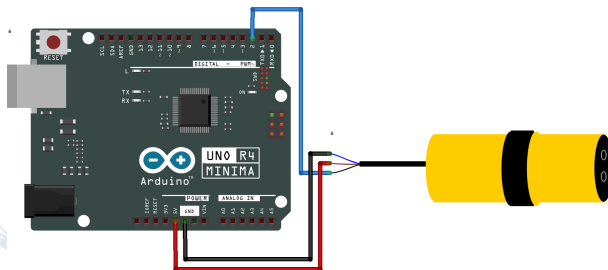
Hinweise über den Sensor III

Anschlussplan

Die Verdrahtung erfolgt direkt an den Pins des **Arduino**:

1 Sensor E18-D80NK:

- **Rote Leitung:** Anschluss an 5V (**Arduino**).
- **Schwarze Leitung:** Anschluss an GND (**Arduino**).
- **Blaue Leitung (Signal):** Anschluss an Digital Pin 2 (**Arduino**).



fritzing

Hinweise über den Sensor IV

Abbildung: Der hier abgebildete Anschlussplan veranschaulicht den Anschluss der Lichtschranke, E18D80NK an dem Arduino Uno rev 4.

Funktion

Ablauf des Funktionstests:

- 1 Upload des Programmcodes via **Arduino IDE**.
- 2 Ruhezustand: Sensorpin auf „HIGH“.
- 3 Objekt in Lichtschranke: Sensor-Ausgang schaltet auf „LOW“.
- 4 Der **Arduino** aktiviert die externe **LED** an Pin 13.
- 5 Objekt entfernt: Mikrocontroller deaktiviert die **LED**.

Beschreibung der Bibliothek Bounce2 I

Die Bounce2-Bibliothek dient zur **Entprellung (Debouncing)** von digitalen Eingängen, insbesondere von mechanischen Tastern und Schaltern. [Fre56]

Problemstellung: Mechanische Schalter erzeugen beim Betätigen **Prellsignale**, d. h. mehrere schnelle Zustandswechsel, die als mehrere Eingaben interpretiert werden können.

Lösung durch Bounce2: Bounce2 filtert diese Prellsignale und stellt sicher, dass:

- nur stabile Zustandsänderungen erkannt werden
- Fehlauslösungen vermieden werden

Hinweise zur Installation I

Um die Bilbiothek verwenden zu können, sollte sie ordnungsgemäß installiert / hinzugefügt werden. Hierfür gibt es zwei Optionen:

- Installation mittels **ArduinoIDE**
- Installation über **GitHub**

Installation mittels ArduinoIDE I

- Öffnen der ArduinoIDE
- Öffnen Reiter „Sketch“
- Öffnen Reiter „Bibliotheken einbinden“
- Öffnen Reiter „Bibliotheken verwalten“
- Im Suchfeld „Bounce2“ eingeben
- Auf „Installieren“ klicken

Nun ist die Bibliothek in das aktuelle Programm eingebunden.

Installation mittels GitHub I

- Öffnen des Repositorys für „Bounce2“ von Thomas Fredericks unter folgendem Link: [Repository](#)
- Herunterladen der aktuellsten Version unter folgendem Link: [Herunterladen](#)
- Komprimierte Datei entpacken
- Ablegen des Ordner: „Bounce2“ in den Ordner „libraries“ im ArduinoIDE Installationsverzeichnis

Funktionen der Bibliothek I

- Überwachung des Eingangszustands über die Zeit
- Verwendung eines **Zeitintervalls (Debounce-Delay)**
- Zustandsänderung wird erst akzeptiert, wenn sie stabil bleibt

Wichtigsten Methoden I

- `attach(pin)`: Verknüpft den Taster mit einem Pin
- `interval(ms)`: Setzt die Entprellzeit
- `update()`: Aktualisiert den Zustand (muss regelmäßig aufgerufen werden)
- `read()`: Gibt den aktuellen stabilen Zustand zurück
- `fell()` / `rose()`: Erkennen von Flanken (Taste gedrückt/losgelassen)

Beispiel der Anwendung der Bibliothek am Tischförderband I

Wir möchten feststellen, ob ein Paket oder Werkstück das Förderband passiert, um einen Motor zu steuern. Die Lichtschranke soll zuverlässig erkennen, ob ein Objekt die Bahn unterbricht, ohne durch Störungen mehrfach ausgelöst zu werden.

Aufbau

- Tischförderband
- Lichtschranke über das Band gespannt
- Arduino zur Auswertung
- Bounce2-Library zur Entprellung

Beispiel der Anwendung der Bibliothek am Tischförderband I

Funktionsweise

- 1 Paket fährt unter die Lichtschranke.
- 2 Lichtstrahl wird unterbrochen, Arduino erkennt LOW.
- 3 Bounce2 verhindert, dass kleine Störungen fälschlicherweise als Paket registriert werden.
- 4 Arduino schaltet Logiksignal am Motortreiber, solange die Lichtschranke unterbrochen ist.

Sicherheitshinweise I

- Betrieb nur mit stabilen 5V DC.
- Signalausgang nicht für hohe induktive Lasten nutzen.
- Optik sauber halten, um Fehlmessungen der **Lichtschanke** zu vermeiden.

Fehlerbehebung I

- **Dauersignal:** Reichweite am Potentiometer reduzieren.
- **Kein Signal:** Pin-Zuweisung in der **Arduino IDE** prüfen.
- **Instabilität:** GND-Verbindung zum **Arduino** sicherstellen.

Kalibrierung I

Um eine Lichtschranke zu kalibrieren ist folgendes Vorgehen empfehlenswert

- 1 Den Sensor gemäß der Herstellerangaben an einen Mikrocontroller anschließen
- 2 Die Lichtschranke montieren, der Abstand zwischen Sender und Empfänger bzw. Sender und Reflektor sollte kleiner als der maximale Abstand gewählt werden.
- 3 Die Lichtschranke ausrichten, sodass Sender & Empfänger coaxial ausgerichtet sind.
Bei Reflex-Lichtschranken sollten Sender und Reflektor coaxial liegen.
- 4 Ein Skript zum Schalten eines Aktors oder ausführen eines Befehls öffnen und auf den Mikrocontroller spielen.

Kalibrierung II

- ⑤ Prüfen ob das gelesene Signal mit Realität übereinstimmt.
in unserem Fall: NC also HIGH-Signal wenn kein unterbrochener Lichtstrahl
- ⑥ Bei falschem Signal: Neuausrichtung.

Vielen Dank
für Ihre Aufmerksamkeit

Quellen

Quellen I

Nachfolgend werden die Quellen der Bilder angegeben, die für diese Präsentation in ihrer ursprünglichen Form oder modifiziert verwendet worden sind.

- [AJ18] Marziyeh Aghaee und Ali Akbar Jalali. "BLDC Motor Speed Control Based on MPC Sliding Mode Multi-Loop Control Strategy". In: *Iranian Conference on Electrical Engineering (ICEE2018)*. Theran, Iran, 2018. DOI: 10.1109/ICEE.2018.8472464.
- [Ali20] Nadien et. al. Ali. "Automatic Speed Control of DC Series Motor by Using Arduino". In: *PRZEGLAD ELEKTROTECHNICZNY* (2020). DOI: 10.15199/48.2023.03.24.
- [Ard24a] Arduino. *Arduino - Pulse Width Modulation (PWM)*. Accessed: 2024-01-30. 2024. URL: <https://docs.arduino.cc/learn/microcontrollers/pwm>.

Quellen II

- [Ard24b] Arduino. *Arduino Digital Pins Documentation*. Accessed: 2024. 2024. URL: <https://www.arduino.cc/en/Tutorial/DigitalPins>.
- [Ard24c] Arduino. *Arduino I2C Communication Guide*. 2024. URL: <https://docs.arduino.cc/tutorials/communication/i2c>.
- [Bar23] Jonathan Bartlett. *Elektronik für Einsteiger: Eine praktische Einführung in Schaltpläne, Schaltkreise und Mikrocontroller*. Springer Berlin Heidelberg, 2023. ISBN: 978-3-662-66242-7.
- [Bas25] Stefan Basler. *Drehgeber und Motor-Feedback-Systeme*. Springer Vieweg Wiesbaden, 2025. ISBN: 978-3-658-49403-2. URL: <https://link.springer.com/book/10.1007/978-3-658-49404-9>.
- [BS22] Massimo Banzi und Michael Shiloh. *Getting started with Arduino*. Maker Media, Inc., 2022. ISBN: 9780596555108.

Quellen III

- [CS14] Scott Chacon und Ben Straub. *Pro Git*. Springer Nature, 2014. ISBN: 9781484200773.
- [Fre56] Thomas Fredericks. *Bounce2 package*. Accessed Date 06.04.2026. 2056. URL: <https://github.com/thomasfredericks/Bounce2>.
- [Gri24] Rudolf Griemert. *Fördertechnik: Auswahl und Berechnung von Elementen und Baugruppen*. Springer Fachmedien, Wiesbaden, 2024. DOI: 9783658433666. URL: <https://link.springer.com/10.1007/978-3-658-43367-3>.
- [HEr23] Gert HEring Ekbert und Schönfelder. *Sensoren in Wissenschaft und Technik*. Springer Vieweg Wiesbaden, 2023. ISBN: 978-3-658-39490-5. URL: <https://link.springer.com/book/10.1007/978-3-658-39491-2>.

Quellen IV

- [HS25] Ankiut Jhinkwan Hemant Sharma Mohit Yadav. "Application of Arduino for Controlling the Speed of DC Motor in Smart Transportation". In: *2025 International Conference on Electronics, AI and Computing (EAIC)*. IEEE. 2025. DOI: 10.1109/EAIC66483.2025.11101414.
- [Ibr23] Dogan Ibrahim. *PID-basierte digitale Regelungstechnik mit Praxisbeispielen für Raspberry Pi und Arduino Uno*. Elektor Verlag GmbH, Aachen., 2023. DOI: 9783895765384.
- [JW14] Robert Resnik Jearl Walker David Halliday. *Fundamentals of Physics*. Wiley, 2014. ISBN: 978-1-118-23072-5.
- [Kli 1] Andreas Klien. "Methods for testing automation and control". In: *The Journal of Engineering* (8-11-2023). DOI: 10.1049/joe.2018.0164. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1049/joe.2018.0164>.

Quellen V

- [lar20] Bindszus lars. *Decentralized control of belt conveyor systems*. Accessed: 2026-03-18. 2020. URL: <https://phi-hannover.de/en/decentralized-control-of-belt-conveyor-systems/>.
- [LM12] Martin Loeffler-Mang. *Optische Sensorik*. Vieweg+Teubner Verlag, Wiesbaden, 2012. DOI: 978-3-8348-1449-4.
- [Luu21] Duc Lich et. al. Luu. "Speed Control and Spacing Control for Autonomous Mobile Robot Platform Equipped with Infrared Sensors". In: *2021 16th International Conference on Engineering of Modern Electric Systems (EMES)*. Location not specified: IEEE, 2021. URL: <https://ieeexplore.ieee.org/document/9484140/>.

Quellen VI

- [McL16] John McLoone Seamus C.; Maloco. "A cost-effective hardware-based laboratory solution for demonstrating PID control". In: *2016 UKACC 11th International Conference on Control (CONTROL)*. Belfast, UK: IEEE, 2016. URL: <http://ieeexplore.ieee.org/document/7737599/>.
- [MM16] Simon Monk und Michael McCabe. *Programming Arduino: Getting Started with Sketches*. Bd. 176. McGraw-Hill's AccessEngineering. New York: McGraw-Hill Education New York, 2016. ISBN: 9781259641633.
- [Nee19] Vijayan Neeraja. "An Arduino based Speed and Rotor Position Measurement Technique for Electric Drives". In: *2019 International Conference on Power Electronics Applications and Technology in Present Energy Scenario (PETPES)*. Mangalore, India, 2019. ISBN: 9781728126555. URL: <https://ieeexplore.ieee.org/document/9003851/>.

Quellen VII

- [Nie15] Clemens Niemeyer. “Motorsteuerung mit Microcontollern”. In: (2015). URL: <https://archive.air.in.tum.de/pub/Main/TeachingWs2014ProseminarMicrocontrollerEmbedded/Motorsteuerung.pdf>.
- [O'D09] Aidan O'Dwyer. *Handbook of PI and PID controller tuning rules*. Bd. 3rd. Imperial College Press, 2009. ISBN: 978-1848162426.
- [Rad16] Roman Radtke. *PID-Tuning für Quadrocopter*. Accessed Date 06.04.2026. 2016. URL: <https://www.heise.de/ratgeber/PID-Tuning-3554159.html>.

Quellen VIII

- [Raj19] Aswinth Raj. *Arduino Nano 33 BLE Sense Review - What's New and How to Get Started?* Accessed Date 15.12.2023. 2019. URL:
<https://circuitdigest.com/microcontroller-projects/arduino-nano-33-ble-sense-board-review-and-getting-started-guide>.
- [RB26] Roboter-Bausatz.de. *Datenblatt - E18-D80NK Infrarot Hinderniserkennung*. Accessed Date 08.04.2026. 2026. URL:
www.roboter-bausatz.de/p/e18-d80nk-infrarot-hinderniserkennung.
- [Rod23] Werner Roddeck. *Einführung in die Mechatronik*. Springer Fachmedien Wiesbaden GmbH, 2023. ISBN: 978-3-658-41949-3. URL:
<https://link.springer.com/book/10.1007/978-3-658-39491-2>.

Quellen IX

- [RPF20] Matthew Sands Richard P. Feynman Robert Leighton.
Feynman Lectures on physics, The Millenium eEition. Basic
books, Hachette Book Group, 2020. ISBN: 978-0-465-02493-3.
- [Ruk24] M.S Aspathi Rukmini Manish Rathi. "Speed Control of DC
Motor using PWM Technique". In: *2024 5th IEEE Global
Conference for Advancement in Technology (GCAT)*. IEEE.
2024. DOI: 10.1109/GCAT62922.2024.10923985.
- [Sau25] Bernd Sauer. *Konstruktionselemente des Maschinenbaus 2:
Grundlagen von Maschinenelementen für Antriebsaufgaben*.
Springer Berlin Heidelberg, 2025. ISBN: 9783662670132.
- [Sch19] Mike Schwager. *EnableInterrupt*. Accessed Date 08.04.2026.
2019. URL: <https://docs.arduino.cc/libraries/enableinterrupt/>.

Quellen X

- [Ten23] TensorFlow. *TensorFlow Lite for Microcontrollers*. Accessed: 14 Jan. 2025. 2023. URL: <https://www.tensorflow.org/lite/microcontrollers>.
- [Wal25] Jörg Wallaschek. *Sensoren und Aktoren*. Springer Berlin Heidelberg, 2025, S. 659–698. ISBN: 9783662670132. DOI: 10.1007/978-3-662-67014-9_9. URL: https://link.springer.com/10.1007/978-3-662-67014-9_9.
- [Wes18] Tim Wescott. “PID Without a PhD”. In: (2018). URL: <https://www.wescottdesign.com/articles/pid/pidWithoutAPhd.pdf>.
- [ZWZ17] Yao Zhang, Wei Wu und Bing Zhao. “Hello Edge: Keyword Spotting on Microcontrollers”. In: *arXiv preprint arXiv:1711.07128* (2017). URL: <https://arxiv.org/abs/1711.07128>.